2026 IEEE 6th International Conference on VLSI Systems, Architecture, Technology and Applications (VLSI SATA) | 979-8-3195-3047-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/VLSISATA69163.2026.11610488

Design and Implementation of High Performance and Low Power RISC-V Processor

Abhinav K R and A R Jayan
Dept. Electronics and Communication Engineering
Government Engineering College Thrissur,
APJ Abdul Kalam Technological University
Kerala, India
abhinav.24m142@gectcr.ac.in, arjayan@gectcr.ac.in

***Abstract*—Modern embedded systems and IoT platforms need processors that give high performance while maintaining low power consumption and high resource efficiency. This work presents the design and implementation of a high performance and low power 32-bit RISC-V processor with 4-stage pipeline for non-load operations and 5-stage pipeline for load operations, also extending the processor as system on chip design to ensure that it can handle real-time data efficiently. The micro-architecture has optimized functional units including Baugh Wooley signed multiplier and 32-bit radix 16 divider. Functional verification of all ISA components, hazard detection, forwarding mechanisms, and branch prediction ensures architectural correctness. The processor is implemented on a Nexys 4 DDR Artix-7 FPGA with a resource requirement of 7415 lookup tables (LUTs) and 3401 flip flops (FFs) and performance is evaluated using Dhrystone 2.1 and CoreMark benchmarks. The design achieves a performance of 1.77 DMIPS/MHz and 3.07 CoreMark/MHz. Different power optimizations are added to the design to reduce dynamic power. The ASIC physical design of the core using Cadence EDA tools shows that the design can reach a maximum of 173.8 MHz in UMC 55nm technology with 0.095179 mm^2 area. Under different PVT conditions the measured dynamic power is 20.86 µW/MHz in worst case corner and 24.59 µW/MHz in typical corner and 32.17 µW/MHz in best case corner. Also, timing analysis is performed to ensure that the design has no timing violations under these cases. The core is extended as an SoC with an MMIO peripheral bridge that integrates IR sensor, PWM motor control, GPIO, UART, seven-segment display, timers, and support for interrupt controller, enabling a complete embedded platform implemented in FPGA.**

***Index Terms*—RISC-V, FPGA, ASIC, processor, benchmark, pipeline**

I. Introduction

The rapid growth of embedded systems and Internet of Things (IoT) applications has increased the requirements for processors that provide high performance while maintaining low power consumption and low hardware complexity. The approach to improve processor performance has depended heavily on CMOS technology scaling, but as scaling approaches its physical limits, the only way to increase speed and efficiency is by designing instruction set architectures (ISA) that can increase performance, reduce power consumption, and meet other requirements. Open source instruction set architecture (ISA) like RISC-V provides a free and flexible platform for implementing custom processor designs made for specific applications while maintaining required design parameters.

This work consists of three main sections FPGA implementation, ASIC physical design, and system on chip (SoC) design around the processor core. Benchmark testing is done on the Nexys 4 DDR Artix 7 FPGA to check the performance of the processor core using Dhrystone and CoreMark benchmark programs. The physical design of the core is done using Cadence Genus and Cadence Innovus and the power, area, and timing reports are obtained. The core is extended as SoC design around the processor core to demonstrate the capability of the core to handle real-time data in FPGA.

II. Related Works

The increase in demand for high performance and low power processors has increased the research on RISC-V micro-architecture. The micro-architecture of [1] has produced benchmark scores of 1.71 DMIPS/MHz (Dhrystone), 4.13 CoreMark/MHz (CoreMark). These are higher than the corresponding values for ARM Cortex-M3 and M4 with low power consumption. This micro-architecture is followed in this work for high performance of the processor core.

RISC-V processor design of [2] offers two configurations (real time and non-real time) and the mode control is done in software to ensure predictability. Real time mode focuses on predictability and non-real time mode focuses on performance. This mode switching really provides good reduction in power consumption. In real time mode, the high performance and power consuming features are disabled ensuring low power consumption and in non-real time mode the processor runs with high performance, which consumes more power.

In the processor design of [3], method of reducing the dynamic power using different clock gating logic has been implemented on the functional units of Pentium 4 processor, resulting in lower dynamic power. Different types of clock gating and their comparison have been done proving that using integrated clock gating cell can highly reduce dynamic power.

Memory mapped input/output (MMIO) design is one of the fastest and simplest memory subsystem designs shown in [4]. To connect with the AVR core, which is implemented on a Xilinx Virtex-4 FPGA, the registers were mapped via WISHBONE slave interface. The processor can access the

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:48:57 UTC from IEEE Xplore. Restrictions apply.

required memory or a peripheral memory, directly by using its address on the bus. The control and status and other features are designed using registers that are mapped in the same memory space.

III. Power Optimization methods

The clock network consumes 50-70 percent of total chip power and this power consumption increases in designs at 55nm and lower technology nodes because dynamic power is directly proportional to the square of voltage, capacitance and clock frequency. The dynamic power (in watts) is given by:

$$P_{\text{dynamic}} = \alpha C V^2 f \tag{1}$$

The frequency (f) of the clock is fixed at 100MHz for this design and the capacitance (C) is fixed. Another way to reduce power is by reducing the switching activity (α).

Clock gating is an important technique used to reduce power consumption and it is implemented by gate level power synthesis tools. RTL clock gating is a commonly used power optimization technique. The challenge of clock gating is to know when and where the gating is to be added without glitch. The goal of clock gating is to control transitions from propagating to different parts of the clock path under provided design conditions [3]. Fig. 1 illustrates clock gating logic where the logic function Fcg controls the clock gating.

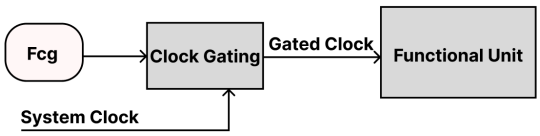


Fig. 1. Clock Gating Principle

The power saving is mainly due to the considerable reduction of switching activity in the logic since when the clock is not active, the unnecessary transitions do not occur [3]. Modern technology libraries and EDA tools support automatic clock gating, with proper attributes and RTL logic EDA tools can identify where the integrated clock gating cell can be inserted. This intelligent clock gating can help to reduce power consumption efficiently [5].

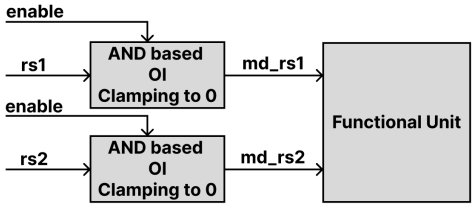


Fig. 2. Operand Isolation Principle

Operand isolation (OI) is another power reduction method in which the inputs of combinational block are gated when output is not used. The AND based (clamping to 0) operand isolation is used as it provides the best trade-off between power saving and area overhead [6]. Operand Isolation is illustrated in Fig. 2 where rs1 and rs2 are inputs and md_rs1 and md_rs2 are the gated rs1 and gated rs2 respectively.

IV. Micro-Architecture

The micro-architecture is designed for increasing efficiency and reducing power consumption. The core architecture of [1] is followed with 1) multiplier modification 2) FSM based divider algorithm and 3) power optimization. The core supports RISC-V ISA base integer instruction set (RV32I) with multiplication and division extension. There are IF: instruction fetch, ID: instruction decode, IE: instruction execute and RW: write back stages in the pipeline. For *non-load* instructions the 4-stage pipeline is used since there is no need for separate memory access as shown in Fig. 3. For *load* instructions, 5-stage pipeline is used since the data memory access and return path take a full cycle [7]. Loads instructions operate as 5-stage with a stage memory read (MEM) comes in between IE and RW as shown in Fig. 4. For *store* instructions the RW stage is skipped as they don't use write back to register files. The block level representation of 4-stage pipeline of the core is shown in Fig. 5.

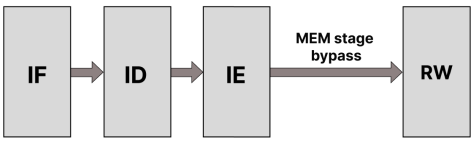


Fig. 3. 4-stage pipeline for *non-load* instructions

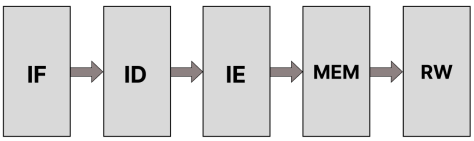


Fig. 4. 5-stage pipeline for *load* instructions

A. Instruction Fetch and Instruction Decode

The IF stage starts in the branch predictor block, it predicts the next program counter (PC) using 2-bit branch history table. 2-bit branch predictor decides whether a branch is to be taken or not before it reaches execute to prevent the instruction fetch from stalling, improving overall instructions per cycle (IPC). 1-bit branch predictor is simple but its accuracy is poor. 2-bit is chosen for increasing accuracy with reduced complexity [1]. The four states are 00 (strongly not taken), 01 (weakly not taken), 10 (weakly taken), and 11 (strongly taken). The unconditional jump instruction JAL uses one cycle since there is no other stage involved, giving fast function calls without any extra bubbles. The fetch outputs including the program counter address and instruction word from instruction cache goes to a fetch buffer which is a register that holds these values for the instruction decode stage [8].

Instruction Decode takes the data from the IF/ID pipeline register which contains the output from fetch stage and it decodes instructions and splits them into opcode, function code

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:48:57 UTC from IEEE Xplore. Restrictions apply.

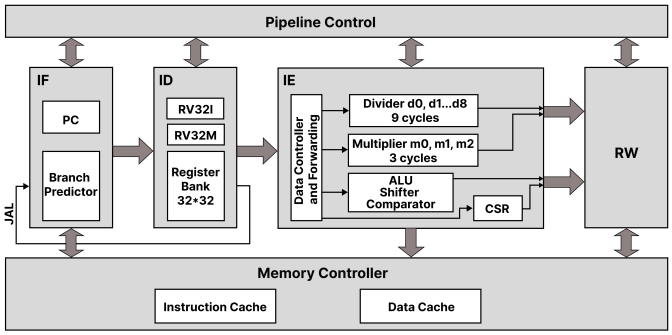


Fig. 5. Pipelining of the micro-architecture

(funct3, funct7), register numbers and immediate counting parts. Using the decode address it requests the first and second source registers of a RISC-V instruction from the 32x32 register bank. RV32I and RV32M decoders help to recognize all the function unit instructions (AND,MUL,SUB,DIV etc..). The output of the decoder stage goes to the decode buffer. The ID/IE pipeline register holds the operands and control signals which are the input for instruction execute stage. In this work, the power optimization logic uses the decode signals to drive the gating networks.

B. Instruction Execute

The data from the ID stage stored in ID/IE register is taken by the IE stage which contains ALU, shifter, comparator, Baugh Wooley Multiplier, radix-16 Divider and control status register (CSR). According to the decode signals, the respective block will operate. The ALU, shifter and comparator functional units have combinational logic design which completes in a single cycle [9]. They have operand isolation, where the block will work only if the decoded function bit indicate an ALU or related instruction. This helps to avoid unwanted transitions in the circuit when the blocks are inactive. This operand isolation is dependent on stalls. If stalls happen in the core, then the same operand isolation will zero the inputs of the functional blocks, stopping unwanted transitions and thus reducing dynamic power consumption. The Execute stage has CSR block used to store status bits, read the built-in counters, real-time ticks, or instructions that the core has run. CSR block is clock gated and is only active when the CSR instructions arrive.

We have used the 33*33 Baugh Wooley signed multiplier [1] which gives the best trade-off between area, power and timing better than other multipliers. In cycle 0, multiplier has operand isolation, and it gets the real operands when a MUL type instruction is active. Each operand is extended by one sign bit and the Baugh Wooley correction row cancels the inverted sign bits, so the 33*33 array still produces the correct signed product.

The partial products are calculated using (2) , the 33 rows of partial products are shifted and aligned and this is divided into three sections bit split, and each section the partial products are added using nine 4-input adders per section (level 1). This data goes to pipeline registers which breaks the critical path and clock gating logic included [10].

$$A * B = a_{n-1}b_{n-1}2^{2n-2} + \sum_{i=0}^{n-2} a_i 2^i \sum_{j=0}^{n-2} b_j 2^j + \sum_{i=0}^{n-2} \overline{a_i b_{n-1}} 2^i + \sum_{j=0}^{n-2} \overline{a_{n-1} b_j} 2^j + 2^n - 2^{2n-1} \quad (2)$$

In cycle 1 the nine added partial products of each section are added using three 4-input adders (level 2), then three added partial products are added using one 3-input adder (level 3). In cycle 2 the first and second sections are combined and combined with third section giving 66 bit result [11].

The divider algorithm used is a 32-bit radix-16 divider which is a four stage FSM. The FSM state diagram of the divider is given in the Fig. 6.

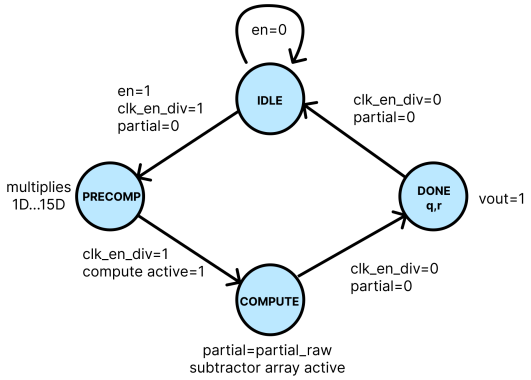


Fig. 6. Radix 16 divider FSM

The first state is the IDLE state in which the divider is clock gated and also has operand isolation, the divider becomes active only if enable signal is high which is dependent on the DIV related instructions. After IDLE state next is the PRECOMP state which computes 1D, 2D....15D in a single cycle where D is the divisor. The circuit elements are active only when the IDLE state is enabled. After the PRECOMP state, the next state COMPUTE gets active. In COMPUTE state, the compare-subtractor array is active which then computes the subtraction, using one-hot select for the largest valid quotient digit avoiding a slow priority chain, thus produces quotient and remainder [12]. In DONE state, the final quotient and remainder are produced and it returns back to IDLE state. The divider takes a total of 9 cycles to produce the outputs. Each state is gated and gets active when one state finishes its processing. So the power consumption is being reduced compared to divider in [1] which is always active with area trade-off. When pipeline asserts a stall signal then execute-stage logic drives the operand-isolation/clock-enable nets low, making unused blocks inactive and saving dynamic power.

C. Memory Stage and Write back Stage

Memory Stage (MEM) is the load/store interface for the pipeline. It accepts the execute stage's load store request that

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:48:57 UTC from IEEE Xplore. Restrictions apply.

contains the byte address, load/store enables, store data, and destination register tag that the MEM stage forwards to the memory controller. When there are no load/store operations, the MEM stage components stay inactive. RW stage ensures that every finished instruction writes to its destination register exactly once, even when multiple functional units complete simultaneously.

V. System on Chip Design

The core is extended as an SoC design to do real-time operations in FPGA, that contains CPU core, memory hierarchy, MMIO peripherals, timer, UART and support for interrupt controller. Fig. 7 shows the block diagram for the design.

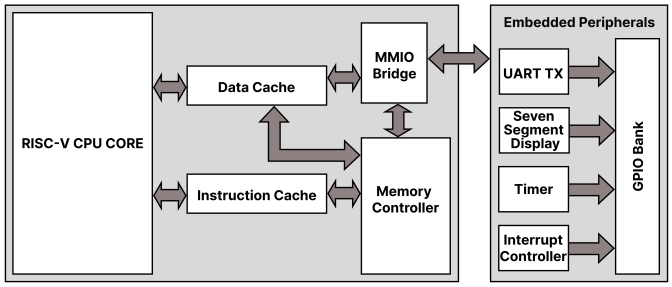


Fig. 7. SoC design

In memory mapped I/O method the input and output of the peripherals are directly mapped to the memory space of the processor. The address bus used for memory access is used to access the peripherals, demanding no special I/O opcodes. This simplifies the design and results in faster hardware, to access the peripherals. The processor handles read or write operations through this peripheral address space [4].

The Nexys 4 DDR Artix-7 FPGA's external pins are designed as GPIO pins for the SoC, making it possible to connect any peripheral devices (like sensors, actuators etc..). Nexys 4 DDR Artix-7 FPGA has an inbuilt seven segment display. In the design, its pins are designed as GPIO outputs which works as same as a seven segment connected through the GPIO pin.

The hardware timer is a 32-bit prescaled countdown timer, which has a prescale stage and a down counter stage. This prescale value is programmable and decides how often the timer ticks. The down counter decides how many of those ticks are generated before completion. The design also has support for an interrupt controller. UART transmission is designed which gives a serial console link between the SoC and an external host [13].

VI. Results

A. FPGA Design of Processor Core

The proposed core is written in Verilog HDL, and the synthesis and implementation of the design are done using Xilinx Vivado on Nexys 4 DDR Artix-7 FPGA. Basic C programs like subtraction, division etc., are simulated using Xilinx Vivado to verify the functionality of the design. Also hazard conditions are also simulated to check if the hazard handling is functioning properly [14]. Instruction cache of size 128 KB and 64 KB data cache are realised in the block RAM of FPGA. The software programs are written in C and converted to hex format using RISC-V GNU Toolchain which is a RISC-V cross compiler. Then this hex file is uploaded as a memory file to the FPGA to execute. After the simulation is verified, the performance of the core is measured using Dhrystone and Coremark benchmark programs. The results of the benchmark are obtained in the PuTTY terminal emulator through the designed UART transmission. The FPGA implementation results show that the core uses 7415 LUTs and 3401 FFs and the timing is met for the core to work properly in 100MHz. The core achieves a performance of 1.77 DMIPS/MHz and 3.07 CoreMark/MHz. The lower number of Coremark/MHz is because of the one extra cycle latency of multiplier compared to the design in [1] but it outperforms most of the cores compared in [1].

B. ASIC Design of Processor Core

The ASIC design of the core is implemented using Cadence EDA tools, and the logic correctness and simulation of the core is done in Xilinx Vivado. Cadence Genus is used for the RTL synthesis of the core and the physical design (floorplanning, placement, clock-tree synthesis, routing, and post-route optimization) is done using the Cadence Innovus tool. The physical design targets the UMC 55nm standard cell library with different PVT conditions; worst case (WC: 1.08V and $125\,^{\circ}\mathrm{C}$), typical case (TC: 1.20V, $25\,^{\circ}\mathrm{C}$) and best case (BC: 1.32V, $-40\,^{\circ}\mathrm{C}$).

TABLE I
Power Report of different PVT corners

Technology Node	Voltage (V)	Temperature (°C)	Power (mW)	Power (μW/MHz)
UMC 55nm (WC)	1.08	125	2.226	22.26
UMC 55nm (TC)	1.20	25	2.499	24.99
UMC 55nm (BC)	1.32	-40	3.285	32.85

TABLE II
Dynamic Power of different PVT corners

Corner	Internal (mW)	Switching (mW)	Leakage (mW)	Dynamic (mW)	Dynamic (μW)	Dynamic (μW/MHz)
WC	1.498	0.5878	0.1405	2.0858	2085.8	20.86
TC	1.899	0.5602	0.0401	2.4592	2459.2	24.59
BC	2.588	0.6293	0.06732	3.2173	3217.3	32.17

The core occupies an area of 0.095179 mm^2. The Table I shows the power obtained in different PVT corners and Table II shows the power breakdown of the power report. In the typical case, the typical case (TC) library represents the nominal PVT, VDD is 1.20V, and the temperature is $25\,^{\circ}\mathrm{C}$. The dynamic power obtained from the typical case is 24.59 μW/MHz. The design operates at 100 MHz clock frequency with a positive slack of 4.250 ns, so the clock frequency can go up to a maximum of 173.8 MHz. In the worst case condition, the worst case (WC) library has lower voltage and higher temperature, VDD is 1.08V and temperature is $125\,^{\circ}\mathrm{C}$

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:48:57 UTC from IEEE Xplore. Restrictions apply.

so the logic is slower and often increases the leakage power. The dynamic power is lower since the VDD is lower, but the timing margin shrinks, so max-delay violations are checked. The dynamic power obtained from the worst case is 20.86 μW/MHz with a leakage of 0.1405 mW and the timing is met with positive setup slack of 0.642 ns and no max-delay (setup) violations. The best case (BC) uses higher voltage and colder temperature where VDD is 1.32V and temperature is $-40\,^{\circ}\mathrm{C}$ so cells switch faster and dynamic power rises proportionally with VDD. BC library helps to check the hold time issues and confirm the design still behaves when everything speeds up. The dynamic power obtained from the best case is 32.17 μW/MHz with a leakage of 0.06732 mW and the hold timing is met for the logic with a positive hold slack of 0.045 ns and no hold violations.

This dynamic power can be reduced further by using lower nanometer low power technology libraries like UMC 40nm [1] which will produce lower area and lower dynamic power consumption.

C. SoC Design around Processor Core

RV32IM SoC was implemented on FPGA with hardware timer and support for interrupt controller. The MMIO bridge controls the dataflow between the processor core and the peripheral devices. The firmware is written in C language which decides the flow of setup, and is converted into a hex format and uploaded as memory file to the FPGA. Simple experiments like showing multiplication results on seven segment display, counting from 0 to 100 etc., are done to validate the correct functioning of the MMIO bridge and other hardware features. To show the real-time operation of the processor, IR sensor

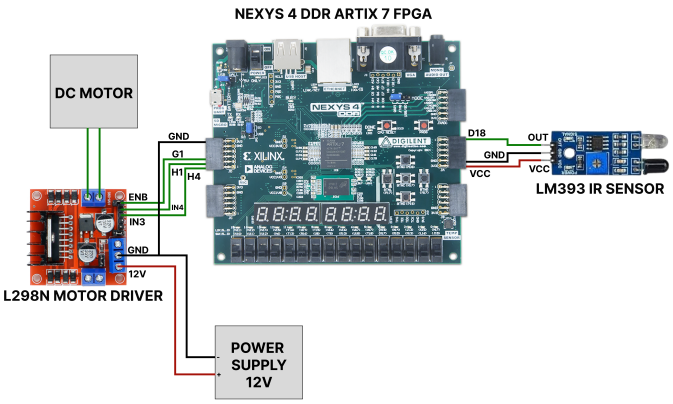


Fig. 8. Circuit diagram of IR sensor motor control

motor control experiment is designed. The peripheral devices are connected to the FPGA as shown in Fig. 8, this contains an IR sensor, motor driver, DC motor and power supply connected to the GPIO ports of the FPGA. The object detection logic is written using C program and converted to hex format and inserted as a memory file for FPGA to execute it. For motor control, the designed PWM controller controls motor speed and direction with two registers. So, when an object is within the IR sensor's detectable range, the sensor detects the obstacle and the seven segment display turns from 0 to 1, and at the same time motor stops rotating. This quick response demonstrates the capability of the processor core to handle real-time data.

VII. Conclusion

This work presents a 32-bit pipelined RISC-V processor core suited for applications with low power. The core supports multiplier, divider and all the integer instruction sets of RISC-V. The core is designed with power optimization to reduce dynamic power consumption as shown through the physical design results and the Dhrystone and CoreMark benchmark results show that the core achieves good performance. The SoC design in FPGA shows that the processor can handle real-time data.

As future work, the core can be implemented to add more RISC-V extensions like RV32A atomics, compressed instructions, or floating-point. The ASIC design using lower nanometer low power libraries like UMC 40nm, UMC 28nm etc., can be implemented to produce lower dynamic power. The SoC design can be extended with proper interrupt controller, DMA-capable MMIO or communication interfaces like UART RX, SPI or CAN.

References

[1] S. Bora and R. Paily, "A High-Performance Core Micro-Architecture Based on RISC-V ISA for Low Power Applications," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 6, pp. 2132–2136, 2021.

[2] M. Poorhosseini and K. Grüttner, "A RISC-V based platform supporting mixed timing-critical and high performance workloads," in *Proc. 26th Euromicro Conference on Digital System Design (DSD)*, 2023, pp. 650–659.

[3] J. Shinde and S. S. Salankar, "Clock gating A power optimizing technique for VLSI circuits," in *Proc. Annual IEEE India Conference*, 2011, pp. 1–4.

[4] R. A. Melo, D. M. Caruso, and S. E. Tropea, "Memory-mapped I/O over dual port BRAM on FPGA," in *Proc. VIII Southern Conference on Programmable Logic*, 2012, pp. 1–6.

[5] S. V and A. Deshpande, "Low Power Implementation of RISC-V Processor with Fine Grained Clock Gating/Power Gating," in *Proc. 9th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS)*, 2025, pp. 1–6.

[6] L. Siddhu, A. Mishra, and V. Singh, "Operand Isolation with Reduced Overhead for Low Power Datapath Design," in *Proc. 27th International Conference on VLSI Design and 13th International Conference on Embedded Systems*, 2014, pp. 483–488.

[7] F. Hussain and S. Sarkar, "Design and FPGA Implementation of Five Stage Pipelined RISC -V Processor," in *Proc. IEEE 9th International Conference for Convergence in Technology (I2CT)*, 2024, pp. 1–6.

[8] T. Zheng, G. Cai, and Z. Huang, "A Soft RISC-V Processor IP with High-performance and Low-resource consumption for FPGA," in *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 2538–2541.

[9] A. Singh, A. Kumar, A. Singh, A. R, and P. K N, "Design and Implementation of RISC-V ISA (RV32IM) on FPGA," *International Journal of VLSI Signal Processing*, vol. 10, pp. 17–21, Jun. 2023.

[10] R. S. Vaidya, V. A. Ingale, A. Phad, and P. Shingare, "Implementation of a high speed low power DSP co-processor based on clock gating and vedic mathematics," in *Proc. International Conference on Communication and Signal Processing (ICCSP)*, 2016, pp. 0836–0839.

[11] I. Patle, "Implementation of Baugh-Wooley Multiplier Based on Soft-Core Processor," *IOSR Journal of Engineering*, vol. 3, pp. 01–07, 2013.

[12] S. Dixit and M. Nadeem, "FPGA Accomplishment of a 16-Bit Divider," *Imperial Journal of Interdisciplinary Research*, vol. 3, no. 2, pp. 2454–1362, Jan. 2017.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:48:57 UTC from IEEE Xplore. Restrictions apply.

[13] C. Shi, Y. Deng, P. Wang, and B. Yan, "A RISC-V-based SoC for pedestrian navigation system," in *Proc. International Conference on Electronic Engineering and Information Systems (EEISS)*, 2024, pp. 123–128.
[14] Z. Qin, "Design and hazard solving of five-stage pipeline RISC-V processor structure," *Applied and Computational Engineering*, vol. 36, pp. 198–203, Feb. 2024.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:48:57 UTC from IEEE Xplore. Restrictions apply.